

Live Video Authenticity Verification — Project Description

BACSE291 Innovative Design Project · Fall 2026 · state as of 5 Oct 2026

1. Problem

Remote identity checks (video KYC, account recovery, exam proctoring) assume the video comes from a real camera filming a real person. Two attacks break this:

- **Injection:** the attacker feeds video into the app without a physical lens: a virtual camera (OBS, v4l2loopback), a hooked capture call inside the app, or an emulator/VM.
- **Synthetic faces:** face swap, reenactment, talking-head or text-to-video generators, usually delivered through an injection channel.

Presentation-attack detection (ISO/IEC 30107-3) looks for masks and screens held up to a lens. It cannot see an injected stream. CEN/TS 18099 (2024) defines injection-attack detection as a separate problem.

2. Approach

Four checks run in sequence on the live camera. Cheap checks run first; any one can end the session.

Gate	Question	How	Cost per tick
1a	Is the source a physical camera in a clean environment?	Kernel device records (V4L2 capabilities, sysfs bus, driver, USB identity); VM, container, debugger, LD_PRELOAD, root and injection-tool checks	~10 ms
1b	Is that camera producing these frames right now?	Random 12-step brightness pattern sent to the physical sensor; frames must follow it	~1.6 s every 10 s
2	Do the frames carry live sensor noise?	Four blind tests on still pixels: fresh noise, no codec block-copying, sensor-like texture, sensor-like colour	~0.35 s
3	Does the face look real?	GenD deepfake detector (WACV 2026, Perception Encoder L) on aligned face crops	~0.37 s

The project's claim: the gates cover each other's blind spots, and the combination detects injected and synthetic video in real time on a laptop without per-camera enrollment.

3. How each gate works

Gate 1a — physical camera (host_integrity.py)

- A node counts as a camera only if its per-node device_caps include video capture, its sysfs entry is not under /sys/devices/virtual, it has a USB/PCI parent, and no layer reports a software-camera driver (v4l2 loopback, akvcam, vivid, ...). The device's reported name is never trusted.
- It builds a camera identity from the USB VID:PID and port, e.g. usb-5986-216a-1-3.
- It BLOCKs on: a debugger or LD_PRELOAD, a VM or container, root, or a non-physical capture node. It FLAGS when software-camera modules or injection tools are present alongside a real camera.

Gate 1b — sensor challenge

- The test sends a random, balanced brightness sequence to the camera with VIDIOC_S_CTRL while reading the stream under test.
- **Statistic:** the median per-pixel brightness shift against the first frame. A moving person doesn't disturb it unless they cover more than half of the image.
- Each frame is labelled with the command active at its driver capture timestamp, which removes queue latency at low frame rates.

- It passes when the partial correlation with the command (controlling for drift) is ≥ 0.90 and the response is ≥ 6 grey levels. The original brightness is always restored.

Gate 2 — live sensor noise (`camera_sensor_noise_profiling.py`)

The test needs no enrollment. It runs on pixels that are still between consecutive frames: a pixel is "still" if its change is small relative to its local gradient, so moving texture can't pose as noise. The stream must pass all four checks:

1. **Noise present:** frame-to-frame noise ≥ 0.6 grey levels. Generators and video codecs largely remove it.
2. **No codec copying:** $\leq 0.1\%$ of 8×8 blocks repeat bit-for-bit. Inter-frame codecs copy whole macroblocks; live sensor noise never repeats a whole block.
3. **Sensor-like texture:** noise neighbour correlation ≥ 0.15 . Demosaicing and JPEG correlate it; added grain is white.
4. **Sensor-like colour:** blue–red noise correlation between 0.30 and 0.97. Grey grain is 1.0 and per-channel grain is 0.

The earlier designs were reference PRNU matching (needs enrollment) and a blind PRNU-presence test (not discriminative). They are kept in the module as a research API and are not used by the app.

Gate 3 — face deepfake model (`deepfake_detector.py`)

- **Pipeline:** YuNet face detection → 5-point alignment exactly as in GenD (1.3× margin, 256 px) → GenD-PE-L → P(fake).
- **Face-quality gate:** a face is scored only if it is ≥ 90 px wide with detection confidence ≥ 0.85 . Below that, the gate abstains.
- **Live decision:** the median P(fake) of the last 3 seconds, flagged at ≥ 0.5 .
- **Model choice:** GenD has the best published cross-dataset AUROC (91.2–91.6% mean over 14 benchmarks).
 - CLIP-L was tested and rejected: it flagged 65% of real webcam windows.
 - DINOv3-L requires Meta's licence approval and was not tested.

Live engine (`live_engine.py`) and app (`app_pages/live.py`)

- A capture thread owns the camera and runs Gate 1b. An analysis thread alternates Gate 2 and Gate 3 every 0.5 s.
- A compute lock pauses analysis during a sensor challenge.
- The camera is released on Stop, when the user leaves the page, when the browser stops polling for 8 s, when another session takes it over, or at exit.
- "Simulate an attack" replaces frames after the driver with a replay, an AI video or an animated photo.
- The server binds to `localhost` only.

4. Results (one laptop webcam, public datasets)

Data:

- 15 webcam sessions on one camera
- VISION phone videos: 11 phone models
- DF40: 48 clips from 12 generators
- TalkingHeadBench: 72 clips from 8 generators
- 16 text-to-video clips (Veo 3, Kling, Hunyuan, Grok)
- 60 CelebV-HQ YouTube clips

Gate	Measurement	Result
1b	Genuine live checks	20/20 pass (r median 1.00)
1b	Replayed frames while the real sensor is challenged	11/11 rejected
1b	Null false passes (non-responding footage, random patterns)	0 / 32,700 (95% bound 9×10^{-5})
1b	10 fps (dim light)	5/5 pass, $r \geq 0.99$
2	Real webcam windows (20 and 45 frames)	104/104 PRESENT
2	Talking-head fakes / DF40 + text-to-video	72/72 and 64/64 ABSENT
2	Grey, colour and blurred synthetic grain on a still photo	18/18 ABSENT
2	x264 replays of webcam sessions	6/6 ABSENT
2	YouTube real clips (encoded)	58/60 ABSENT (by design: not a live stream)
3	Real webcam windows with a clear face	0/11 flagged (max P 0.18)
3	Fakes caught at $P \geq 0.5$ (single 16-frame decision)	69%
3	YouTube reals flagged at $P \geq 0.5$	16%
3	AUROC: webcam real vs fakes / all real vs fakes	0.96 / 0.865
3 (old heuristic)	Fakes caught	2/64 (replaced)

Cost on the test laptop (RTX 4050):

- Gate 2: 348 ms per tick; Gate 3: 365 ms per tick
- Process memory: ~1.5–1.9 GB resident
- "17.8 GB" in task managers is virtual address space reserved by CUDA, not RAM in use

5. Limitations

- **One camera, one room.** All genuine data, and every threshold, comes from this laptop's webcam, so thresholds were tuned and tested on the same data. A publishable study needs ≥ 10 webcams, several people, varied lighting and a held-out test split.
- **No real virtual camera was tested.** v4l2loopback is not installed. Gate 1a was verified on a replica of its sysfs layout.
- **Adaptive attackers:**
 - Someone who reads the camera's brightness setting could modulate injected frames to match.
 - Someone who injects *raw* frames with realistic sensor-like noise could pass Gate 2.
 - A replay of raw frames from this same camera passes Gate 2; Gate 1b is what stops it.
- **Kernel-level compromise is out of scope.**
- **Gate 3** catches about 69% of fakes per decision and flags 16% of compressed YouTube faces. It is one signal, not a proof.
- **Platform:** Linux/V4L2 only. The Android SDK (`app/`) is a separate Layer 1 prototype; its JSON verdict is accepted but it does not stream frames.

6. Open issue (unresolved)

In the last browser test of the app, the engine's background thread printed "Loading the deepfake model..." and did not finish within 45 s, so Gates 2 and 3 stayed on "warming up". The same engine works when the model is loaded before the engine starts (headless test: all gates reporting, liveness $r = 1.00$).

Suspected cause: loading the model inside the analysis thread in the Streamlit process. Debugging was interrupted. **The changes since commit `d8bb0c1` (blind Gate 2, GenD Gate 3, plain-language UI, performance fixes) are not yet committed.**

7. Run

```
source .venv/bin/activate
streamlit run app.py           # live page; Start → checks run continuously
python run_tests.py           # 58 tests
python benchmarks/run_benchmark.py  # Gate 1b / reference-PRNU benchmark
```

Gate 3 needs PyTorch (CUDA build for NVIDIA GPUs) and downloads GenD-PE-L (~1.2 GB) on first use.

8. Repository

Path	Purpose
host_integrity.py	Gate 1a + 1b
camera_sensor_noise_profiling.py	Gate 2 (live noise; optional reference PRNU)
deepfake_detector.py	Gate 3 (YuNet + GenD)
pipeline.py	Gate orchestration for files and one-shot live sessions
live_engine.py , app_pages/live.py	Real-time engine and live page
ui.py , app_pages/analyze_file.py , app_pages/device.py	File analysis and device pages
temporal_consistency_analysis.py	Original Gate 3 heuristic (kept, no longer used)
benchmarks/	Dataset fetchers, capture script, benchmark runner, results
docs/ARCHITECTURE.md	Long-form design history and earlier measurements
app/	Android Layer 1 prototype

9. Next steps

1. Fix the in-app model-loading stall, then commit.
2. Collect multi-camera, multi-person data. Re-tune Gate 2 and Gate 3 thresholds on a training split and report on a held-out split.
3. Run real attacks: v4l2loopback + OBS, DeepFaceLive, and an LD_PRELOAD frame swap.
4. Move the challenge and verification server-side (server-issued pattern, signed client attestation) so a compromised client cannot skip the checks.
5. Test the adaptive attacks in §5.